

Introducción a los patrones de diseño

Introducción a los patrones

¿Para qué
sirven?

Identificar problemas típicos y sus soluciones adecuadas.

Patrones de software

Los patrones de software son formas “estandarizadas” de resolver problemas comunes de diseño en el desarrollo de software.

Las ventajas del uso de patrones son:

- Conforman un amplio catálogo de problemas y soluciones.
- Estandarizan la resolución de determinados problemas.
- Condensan y simplifican el aprendizaje de las buenas prácticas.
- Proporcionan un vocabulario común entre desarrolladores.
- Evitan “reinventar la rueda”.

Definiciones y propósito



Un patrón es una solución para un problema en un contexto.



Su propósito es una **reutilización eficiente**.

Características de los patrones de software

01

Nombre del patrón

(Una de las partes más difíciles).
Un nombre que sirva para identificar el patrón y que nos permita individualizar su objetivo.

02

Problema

Descripción de cuándo utilizarlo.

03

Solución

Detalle de la solución que se obtendrá si se aplica este patrón.

04

Consecuencias (buenas y malas)

Lista de las ventajas y desventajas de la aplicación de este patrón.

Clases de patrones



Patrones **sobre creación**

Cómo crear objetos.
(Factoría abstracta, builder, prototipo, singleton).



Patrones **estructurales**

Cómo agrupar y organizar objetos.
(Bridge, adapter, proxy, facade, flyweight).



Patrones **de comportamiento**

Cómo se relacionan en ejecución grupos de objetos.
(Command, interpreter, iterator, mediator, observar)

Patrones sobre creación

Se enfrenta al problema de la instanciación.

Muchas veces los constructores no son suficientes:

- Delegar la creación a otros objetos.
- Generalizar y encapsular detalles para permitir su re-uso.
- Flexibilizar quién crea, qué produce, cómo es creado y cuándo.

Patrones estructurales



Estudian cómo se relacionan los objetos en tiempo de ejecución (mapa del sistema).



Sirven para diseñar las interconexiones entre objetos.

Patrones de comportamiento

Tiene que ver con la dimensión temporal.

Estudia las relaciones entre llamadas entre los diferentes objetos.

Incide en facilidades para tiempo de ejecución.

Implementación de los patrones utilizando computadoras

En la implementación de un patrón Singleton solo puede haber una instancia:

```
Class Singleton {  
  
    public:  
  
    static Singleton * instance (void);  
  
    void DoSomething (int);  
  
    protected:  
  
    singleton ();  
  
    private:  
  
    static Singleton * Sinstance;  
  
};
```



© Universidad de Palermo

Prohibida la reproducción total o parcial de imágenes y textos.